



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

ASSESSMENT TOOL 1

APPLICATION BASED QUESTIONS

NAME: JANA LAHARI

REG NO:192424224

COURSE NAME: COMPUTER ARCHITECTURE

COURSE CODE: CSA1205

Q1. Smart City Surveillance System

A smart city surveillance system continuously receives video feeds from multiple cameras installed at roads, public places, and buildings. These video streams are transferred from I/O devices (cameras) to main memory, where they are temporarily stored. The CPU then fetches the video data from memory, processes it using threat detection algorithms, and generates alerts if suspicious activities are detected. During peak hours, when many cameras send data simultaneously, the system experiences delays because of the heavy interaction between the CPU, memory, and I/O devices.

Interaction between CPU, Memory, and I/O

- I/O Devices (Cameras): Capture live video and send large amounts of data to the system.
- Memory (RAM): Stores video frames temporarily and provides data to the CPU.
- CPU: Fetches instructions and video data from memory, processes each frame, and identifies potential threats.

Since all these components communicate continuously, the CPU may spend more time waiting for data from memory or I/O devices. This creates CPU bottlenecks, increases processing time, and results in lag during real-time surveillance.

Simple Flow Diagram

CCTV Cameras (I/O)



Main Memory



CPU



Threat Detection



Alert Generation

Single Bus vs Multi-Bus Structure

Single Bus Structure

In a single bus system, the CPU, memory, and I/O devices share the same communication bus.

Advantages:

- Simple design
- Low cost
- Easy to implement

Disadvantages:

- Only one data transfer occurs at a time.
- High bus contention during heavy traffic.
- CPU has to wait for memory or I/O access.
- Lower overall performance.

Effect: During peak hours, multiple cameras compete for the same bus, causing delays in video processing and threat detection.

Multi-Bus Structure

In a multi-bus system, separate buses are provided for communication between CPU, memory, and I/O devices.

Advantages:

- Multiple data transfers occur simultaneously.
- Reduced waiting time.
- Higher bandwidth.
- Faster communication between system components.
- Better performance under heavy workloads.

Disadvantages:

- Higher implementation cost.
- More complex hardware design.

Effect: The CPU can process video data more efficiently, reducing lag and improving real-time threat detection.

Comparison Table

Feature	Single Bus	Multi-Bus
Number of Buses	One	Multiple
Data Transfer	One at a time	Simultaneous
Speed	Slow	Fast
Bus Contention	High	Low
Performance	Lower	Higher

Improving the Instruction Execution Cycle

The CPU executes every instruction through the following stages:

1. Fetch – Retrieve the instruction from memory.

2. Decode – Interpret the instruction.
3. Execute – Perform the required operation.
4. Memory Access – Read or write data if needed.
5. Write Back – Store the final result.

To improve system performance, the following architectural enhancements can be used:

- Instruction Pipelining: Executes multiple instructions simultaneously, increasing throughput.
- Cache Memory: Stores frequently used data, reducing memory access time.
- Direct Memory Access (DMA): Allows I/O devices to transfer data directly to memory without CPU intervention.
- Multi-Core Processors: Distribute video processing tasks across multiple CPU cores.
- Branch Prediction: Reduces delays caused by conditional instructions.

These techniques decrease CPU waiting time, reduce execution time, and improve overall performance.

Conclusion

The lag in the smart city surveillance system is mainly caused by heavy communication between the CPU, memory, and I/O devices, leading to bus contention and increased CPU waiting time. A multi-bus architecture provides faster and parallel data transfer compared to a single-bus system, making it more suitable for handling multiple video streams. Additionally, improvements such as instruction pipelining, cache memory, DMA, branch prediction, and multi-core processors optimize the instruction execution cycle, resulting in faster threat detection, improved system responsiveness, and efficient real-time surveillance.

Q2. Real-Time Stock Trading Application

A real-time stock trading application processes millions of transactions every second. The processor continuously executes instructions to analyze market data, update stock prices, and complete buy or sell orders. During high load, users experience delays because the processor has a high CPI (Cycles Per Instruction) due to frequent memory accesses and branch

instructions. A high CPI increases the time required to execute instructions, reducing the overall performance of the system.

Fetch–Decode–Execute Cycle

Every instruction executed by the CPU passes through the following stages:

1. Fetch: The CPU fetches the instruction from main memory.
2. Decode: The Control Unit interprets the instruction and identifies the operation to be performed.
3. Execute: The Arithmetic Logic Unit (ALU) performs the required calculation or logical operation.
4. Memory Access: If necessary, the CPU reads from or writes data to memory.
5. Write Back: The result is stored in a register or memory.

Instruction Cycle Diagram

Fetch



Decode



Execute



Memory Access



Write Back

Impact of the Fetch–Decode–Execute Cycle on Execution Time

In a stock trading application, millions of instructions are executed every second. During high load:

- Frequent memory access increases the time taken to fetch data.

- Branch instructions cause the processor to wait until the next instruction is determined.
- Cache misses force the CPU to access slower main memory.
- As a result, the CPI increases, leading to slower transaction processing and delayed responses.

The CPU execution time is calculated as:

$$\text{CPU Execution Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

A higher CPI directly increases the execution time and reduces system performance.

Methods to Reduce CPI

1. Cache Memory

- Stores frequently used instructions and data.
- Reduces memory access time.
- Improves execution speed.

2. Instruction Pipelining

- Allows multiple instructions to be processed simultaneously.
- Increases CPU utilization and throughput.
- Reduces overall execution time.

3. Branch Prediction

- Predicts the outcome of branch instructions before execution.
- Reduces pipeline stalls.
- Improves processing speed.

4. Multi-Core Processors

- Distributes multiple trading requests across different CPU cores.
- Enables parallel processing.
- Handles high workloads efficiently.

5. Out-of-Order Execution

- Executes independent instructions while waiting for slower instructions to complete.
- Minimizes CPU idle time.
- Improves overall performance.

6. High-Speed Memory

- Faster RAM and larger cache reduce memory access delays.
- Improves data transfer between CPU and memory.

Architectural Enhancements

Enhancement	Benefit
Cache Memory	Reduces memory access time
Instruction Pipelining	Executes multiple instructions simultaneously
Branch Prediction	Minimizes delays due to branch instructions
Multi-Core Processor	Supports parallel execution of transactions
Out-of-Order Execution	Reduces CPU idle time
High-Speed Memory	Improves memory performance

Conclusion

The delays in the real-time stock trading application are mainly caused by a high CPI, frequent memory accesses, and branch instructions. These factors increase the execution time during the fetch–decode–execute cycle, resulting in slower transaction processing. By implementing cache memory, instruction pipelining, branch prediction, multi-core processors, out-of-order execution, and high-speed memory, the CPI can be reduced significantly. These architectural enhancements improve CPU performance, reduce execution time, and ensure faster and more efficient stock trading operations during high system load.

Q3. Embedded System Based on 8085 Microprocessor

An embedded system based on the 8085 microprocessor is used in an industrial temperature control unit. Sensors continuously measure the temperature and send the data to the microprocessor. The processor reads the sensor values, compares them with the required temperature, and controls actuators such as heaters or cooling fans. If the program uses incorrect addressing modes, wrong data may be accessed, leading to incorrect temperature readings and improper operation of the system.

Addressing Modes in 8085

Addressing modes specify how the processor accesses data or operands. The commonly used addressing modes in the 8085 microprocessor are:

1. Immediate Addressing Mode

- The data is provided directly in the instruction.
- No memory access is required.

Example:

MVI A, 25H

Here, 25H is directly loaded into register A.

2. Register Addressing Mode

- Data is stored in CPU registers.
- Faster because no memory access is involved.

Example:

MOV A, B The contents of register B are copied to register A.

3. Direct Addressing Mode

- The memory address of the data is specified in the instruction.

Example:

LDA 2500H

The data stored at memory location 2500H is loaded into register A.

4. Register Indirect Addressing Mode

- The memory address is stored in a register pair (HL).

Example:

MOV A, M

The processor reads data from the memory location pointed to by the HL register pair.

5. Implied Addressing Mode

- The operand is implied by the instruction itself.

Example:

CMA

This instruction complements the contents of register A.

Influence of Addressing Modes on Data Access

The choice of addressing mode directly affects how the processor accesses sensor data.

- Immediate addressing is suitable for fixed values such as threshold temperatures.
- Register addressing provides faster execution for temporary calculations.
- Direct addressing is useful when sensor values are stored at fixed memory locations.
- Register indirect addressing is suitable when multiple sensor readings are stored in consecutive memory locations.
- Implied addressing is mainly used for simple operations on registers.

Selecting the correct addressing mode ensures that the processor reads the correct sensor value and performs accurate temperature control.

Possible Causes of Incorrect Temperature Readings

The system may process incorrect temperature values due to:

- Accessing the wrong memory location.
- Incorrect initialization of the HL register pair.
- Using an inappropriate addressing mode.

- Accidental overwriting of sensor data.
- Programming errors in data transfer instructions.

These issues may cause the processor to compare incorrect values and activate the heater or cooling system unnecessarily.

Effective Use of the 8085 Instruction Set

The following instructions help ensure accurate and reliable operation:

Instruction	Purpose
LDA	Load sensor data from memory
STA	Store processed data in memory
MOV	Transfer data between registers
MVI	Load immediate values
LXI	Load register pair with memory address
CMP	Compare temperature values
JZ / JNZ	Make decisions based on comparison results
IN / OUT	Read data from sensors and send signals to actuators

Using these instructions correctly ensures proper data transfer, comparison, and control of industrial equipment.

Conclusion

In an 8085-based industrial temperature control system, the correct use of addressing modes is essential for accurate data access and reliable operation. Incorrect addressing may result in wrong temperature readings and improper control of heaters or cooling devices. By selecting appropriate addressing modes and effectively using the 8085 instruction set such as LDA, MOV, MVI, CMP, JZ, and IN/OUT, the system can process sensor data accurately, improve reliability, and ensure efficient industrial temperature control.

Q4. 8086 Segmented Memory System

The 8086 microprocessor uses a segmented memory architecture to access more memory efficiently. Memory is divided into different segments such as Code Segment (CS), Data Segment (DS), Stack Segment (SS), and Extra Segment (ES). This organization allows multiple programs to execute simultaneously and improves memory management. However, if the segment registers are not handled correctly, it can lead to incorrect data access, stack overflow, and program execution errors.

Segmentation in 8086

The 8086 has a 20-bit address bus, allowing it to access 1 MB (2^{20} bytes) of memory. Since its registers are only 16 bits, it uses segment and offset addresses to generate the physical address.

Physical Address Formula

Physical Address = Segment Address $\times$ 10H + Offset Address

For example:

- Segment Address = 2000H
- Offset Address = 1500H

Physical Address = $2000H \times 10H + 1500H = 21500H$

Types of Segments

1. Code Segment (CS)

- Stores the program instructions.
- Works with the Instruction Pointer (IP).
- The CPU fetches instructions from this segment during execution.

2. Data Segment (DS)

- Stores variables, constants, and data used by the program.
- Most data access instructions use the DS register.

3. Stack Segment (SS)

- Stores temporary data, return addresses, and local variables.

- Works with the Stack Pointer (SP).
- Used during function calls and interrupts.

4. Extra Segment (ES)

- Provides additional memory for storing data.
- Mainly used in string processing operations.

Problems Due to Improper Segment Handling

1. Incorrect Data Access

If the Data Segment (DS) register contains an incorrect value, the CPU reads data from the wrong memory location. This may result in:

- Incorrect program output.
- Data corruption.
- Unexpected program behavior.

2. Stack Overflow

The stack grows downward in memory. If too many PUSH instructions are executed without corresponding POP instructions, or if the Stack Segment (SS) or Stack Pointer (SP) is not initialized correctly, the stack may overflow.

This can cause:

- Program crashes.
- Loss of return addresses.
- Execution errors.

3. Wrong Code Execution

If the Code Segment (CS) register is incorrect, the CPU fetches instructions from the wrong location, leading to incorrect execution or program failure.

Instruction Execution in 8086

The CPU executes instructions through the following stages:

1. Fetch – The instruction is fetched from the Code Segment.
2. Decode – The Control Unit interprets the instruction.
3. Execute – The ALU performs the required operation.
4. Store Result – The result is written to a register or memory.

Addressing Modes Used in 8086

Some commonly used addressing modes are:

- Immediate Addressing – Data is part of the instruction.
- Register Addressing – Data is stored in registers.
- Direct Addressing – Memory address is specified directly.
- Register Indirect Addressing – Memory address is stored in a register.
- Based and Indexed Addressing – Used for accessing arrays and strings efficiently.

Using the appropriate addressing mode ensures that the processor accesses the correct data and executes instructions accurately.

Methods to Ensure Correct Program Execution

- Initialize CS, DS, SS, and ES registers properly before program execution.
- Use correct addressing modes for memory access.
- Balance every PUSH instruction with a corresponding POP instruction.
- Maintain proper stack management to prevent overflow.
- Verify segment and offset addresses before accessing memory.

Conclusion

The 8086 segmented memory architecture improves memory organization by dividing memory into Code, Data, Stack, and Extra Segments. However, incorrect handling of segment registers can cause wrong data access, stack overflow, and execution errors. Proper initialization of segment registers, careful stack management, and the correct use of addressing modes ensure accurate instruction execution and reliable operation of multiple programs in the 8086 microprocessors.

Q5. Number System Conversion and CPU Performance

In a data communication system, packet information is often represented in hexadecimal (Hex) format because it is shorter and easier to read than binary. However, the CPU processes all instructions and data in binary. Therefore, correct conversion between hexadecimal and binary is essential. If a software engineer incorrectly converts these values, the CPU may interpret the data wrongly, resulting in incorrect instruction execution and data processing errors.

Importance of Hexadecimal to Binary Conversion

Hexadecimal is widely used because one hexadecimal digit represents 4 binary bits.

Example:

Hexadecimal	Binary
A	1010
F	1111
3F	0011 1111

Using hexadecimal makes debugging, memory addressing, and data representation simpler while the CPU internally converts it into binary for processing.

Effects of Incorrect Conversion

If hexadecimal values are converted incorrectly, several problems may occur:

- The CPU may execute the wrong instruction.
- Incorrect memory addresses may be accessed.
- Packet data may become corrupted.
- Communication errors may occur between devices.
- System reliability and security may be affected.

Example:

Correct Conversion:

3F → 0011 1111

Incorrect Conversion:

3F → 0011 1011

The incorrect binary value changes the actual data, leading to incorrect processing by CPU.

Effect on Instruction Execution

The CPU executes instructions through the following stages:

1. Fetch – Retrieves the instruction from memory.
2. Decode – Converts the instruction into machine operations.
3. Execute – Performs the required operation.
4. Store Result – Saves the output in memory or registers.

If the binary value obtained from hexadecimal conversion is incorrect, the CPU may fetch or decode the wrong instruction, resulting in incorrect execution and system errors.

Efficient CPU Design to Minimize Errors

Modern processors include several architectural features to improve accuracy and performance:

- Cache Memory: Reduces memory access time.
- Error Detection and Correction (ECC): Detects and corrects memory errors.
- Instruction Pipelining: Executes multiple instructions simultaneously to improve speed.
- High-Speed ALU: Performs arithmetic and logical operations efficiently.
- Efficient Control Unit: Ensures proper instruction decoding and execution.

These features help reduce processing errors and improve the reliability of real-time systems.

Role of Benchmarking

Benchmarking is the process of measuring CPU performance using standard tests. It helps identify performance issues and evaluate the efficiency of a processor.

Common benchmarking parameters include:

- CPI (Cycles Per Instruction) – Measures the average number of clock cycles required to execute an instruction.
- Execution Time – Time taken to complete a program.
- Clock Speed – Number of cycles executed per second.
- Throughput – Number of tasks completed in a given time.
- MIPS (Million Instructions Per Second) – Measures instruction processing speed.

If benchmarking shows high execution time or high CPI, developers can identify problems such as incorrect data conversion or inefficient instruction execution and optimize the system accordingly.

Conclusion

Hexadecimal-to-binary conversion plays a vital role in data communication systems because the CPU processes all instructions in binary format. Incorrect conversion can lead to wrong instruction execution, memory access errors, and data corruption. Efficient CPU design, including cache memory, error detection mechanisms, instruction pipelining, and a high-speed control unit, helps improve processing accuracy. Additionally, benchmarking techniques such as CPI, execution time, and MIPS enable developers to detect performance issues and ensure reliable and efficient operation of real-time communication systems.